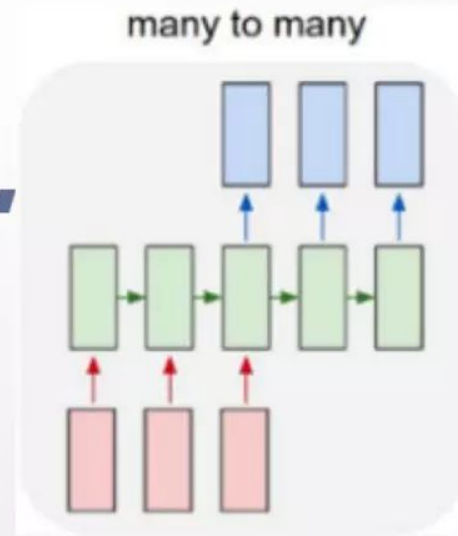
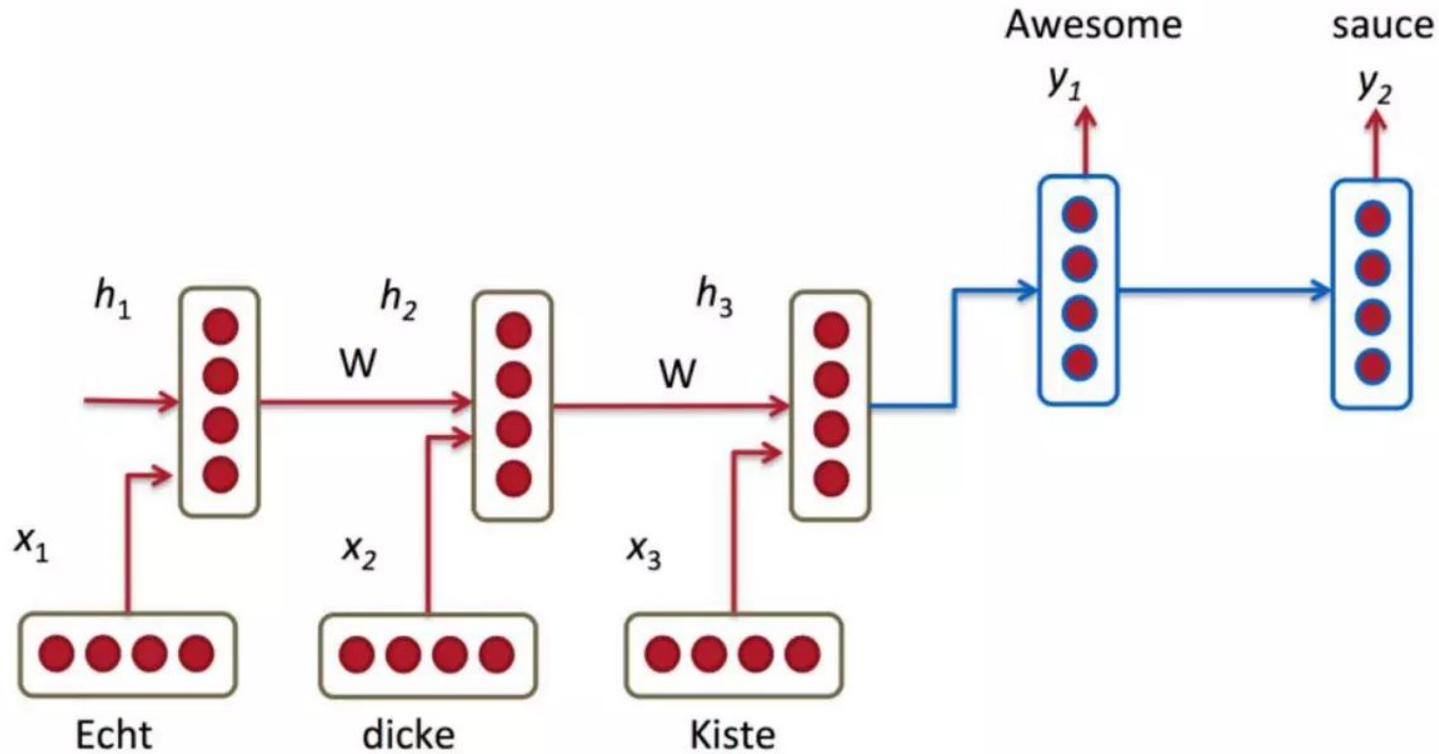




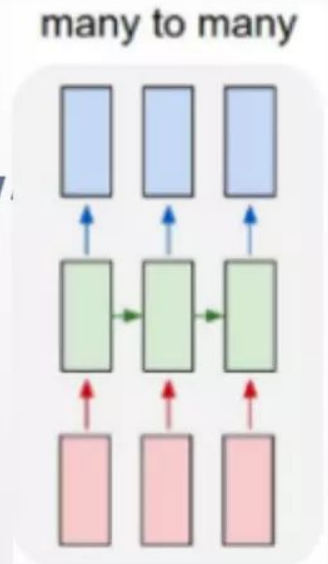
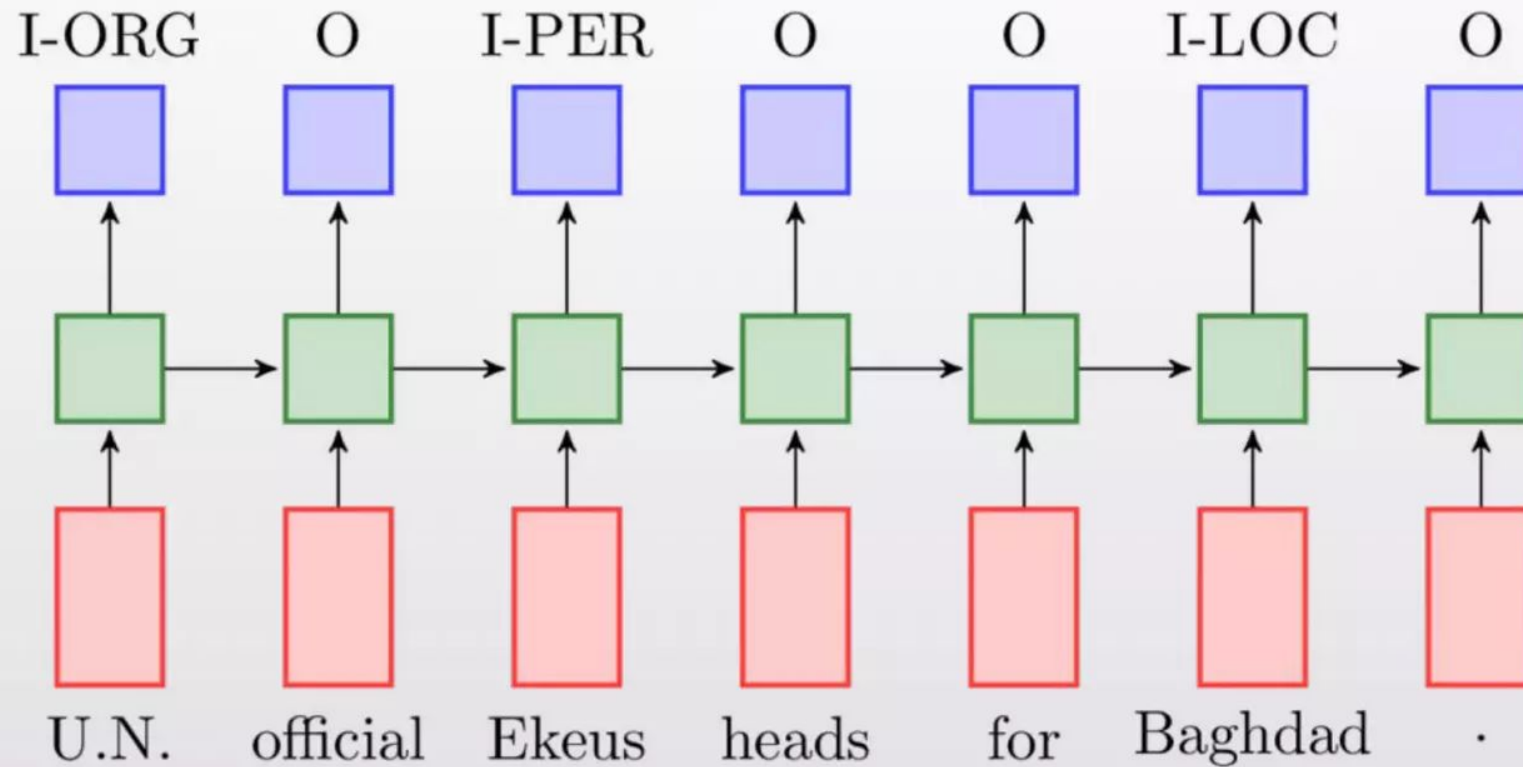
Recurrent Neural Network

RNN untuk Mesin Translasi Jerman-Inggris



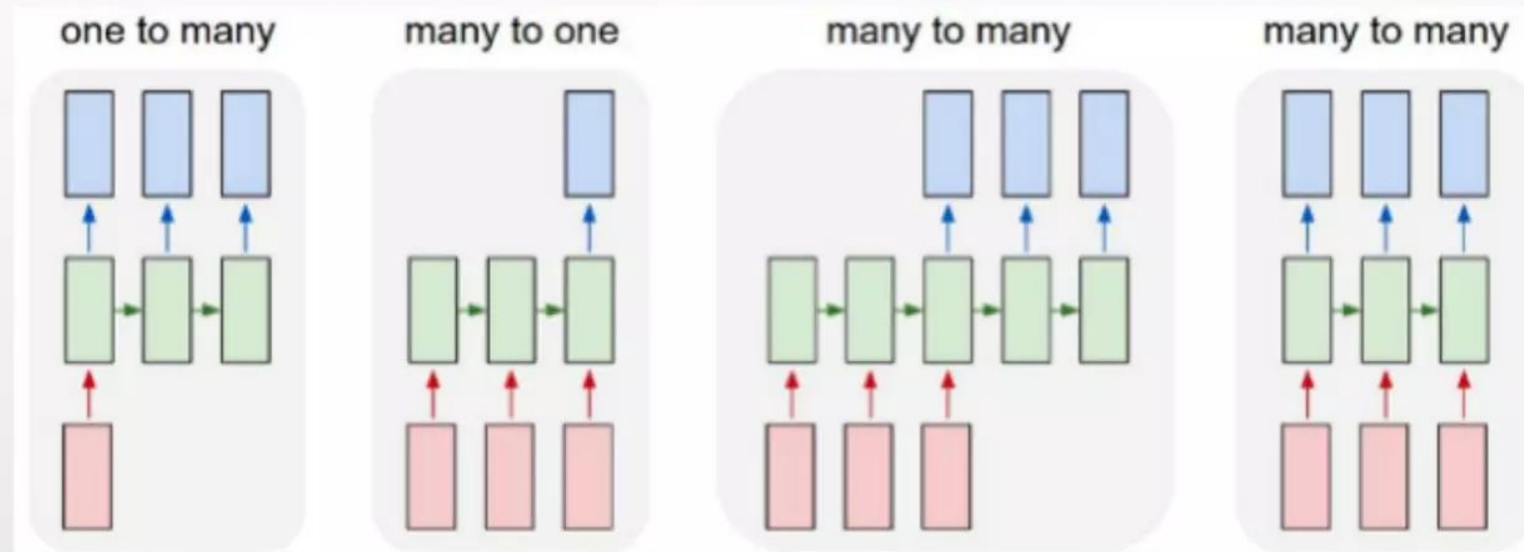
- Terjemahan Mesin: input adalah urutan kata dalam bahasa sumber (misalnya bahasa Jerman). Output adalah urutan kata dalam bahasa target (misalnya bahasa Inggris).
- Perbedaan utama adalah bahwa output kita hanya dimulai setelah kita melihat input yang lengkap, karena kata pertama dari kalimat yang kita terjemahkan mungkin memerlukan informasi yang diambil dari urutan input yang lengkap.

RNN untuk Sequence Tagging



- Input adalah urutan kata, dan output adalah urutan tag POS untuk setiap kata.

Recurrent Neural Network: Analisis Sentimen



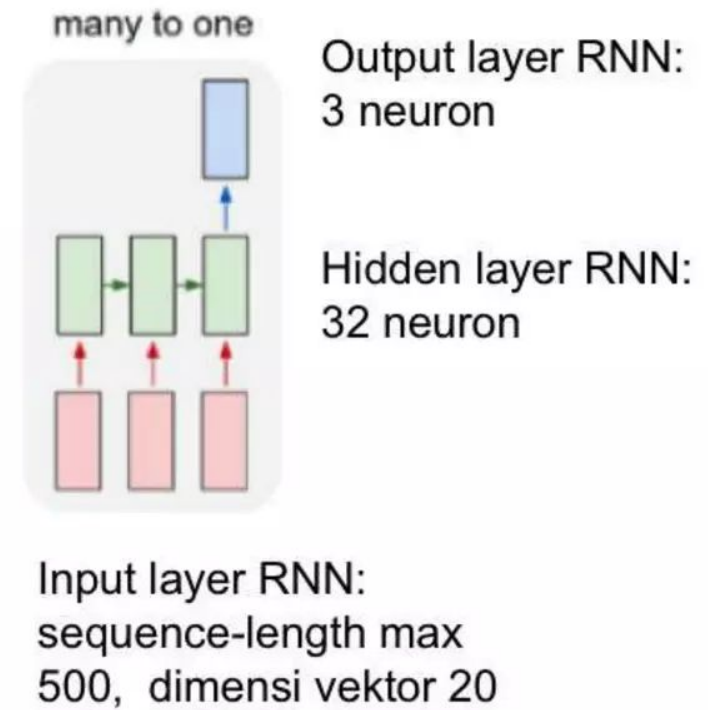
- Klasifikasi sentimen level kalimat:
 - ❑ one to one (without RNN, fixed-sized input vector $x_t \rightarrow$ fixed-sized output vector o_t), atau
 - ❑ many to one (sequence of fixed-sized input vector $x_t \rightarrow$ fixed-sized output vector o_t)
- Klasifikasi sentimen level aspek:
 - ❑ Many to many (synced sequence of fixed-sized input vector x_t and o_t)

Contoh RNN pada Keras

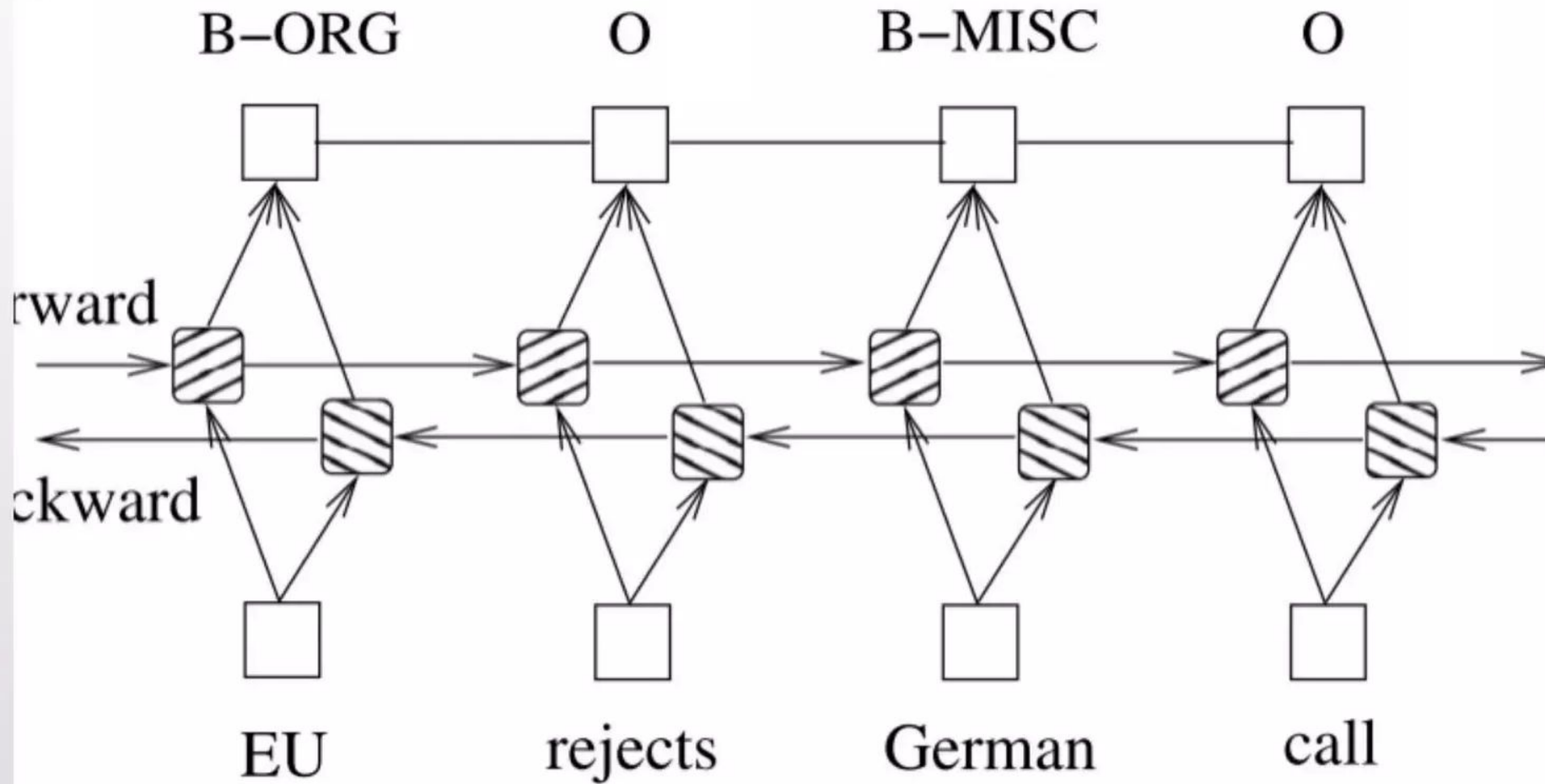
```
from keras.preprocessing import sequence
from keras.layers import Embedding
from keras import models
from keras import layers
model = models.Sequential()
```

```
maxlen = 500
embed_dim = 20
```

```
model.add(Embedding(jum_vocab+1, embed_dim, input_length=maxlen))
model.add(layers.SimpleRNN(32))
model.add(layers.Dense(3, activation='softmax'))
model.compile(optimizer="adam",
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```



Bidirectional RNNs: Ekstraksi Informasi



Bidirectional RNNs: BPTT

for $t = 1$ to T do

Do forward pass for the forward hidden layer, storing activations at each timestep

for $t = T$ to 1 do

Do forward pass for the backward hidden layer, storing activations at each timestep

for $t = 1$ to T do

Do forward pass for the output layer, using the stored activations from both hidden layers

Algorithm 3.1: BRNN Forward Pass

for $t = T$ to 1 do

Do BPTT backward pass for the output layer only, storing δ terms at each timestep

for $t = T$ to 1 do

Do BPTT backward pass for the forward hidden layer, using the stored δ terms from the output layer

for $t = 1$ to T do

Do BPTT backward pass for the backward hidden layer, using the stored δ terms from the output layer

Algorithm 3.2: BRNN Backward Pass

Deep Recurrent Networks

Deep recurrent networks:
multiple hidden layers RNN

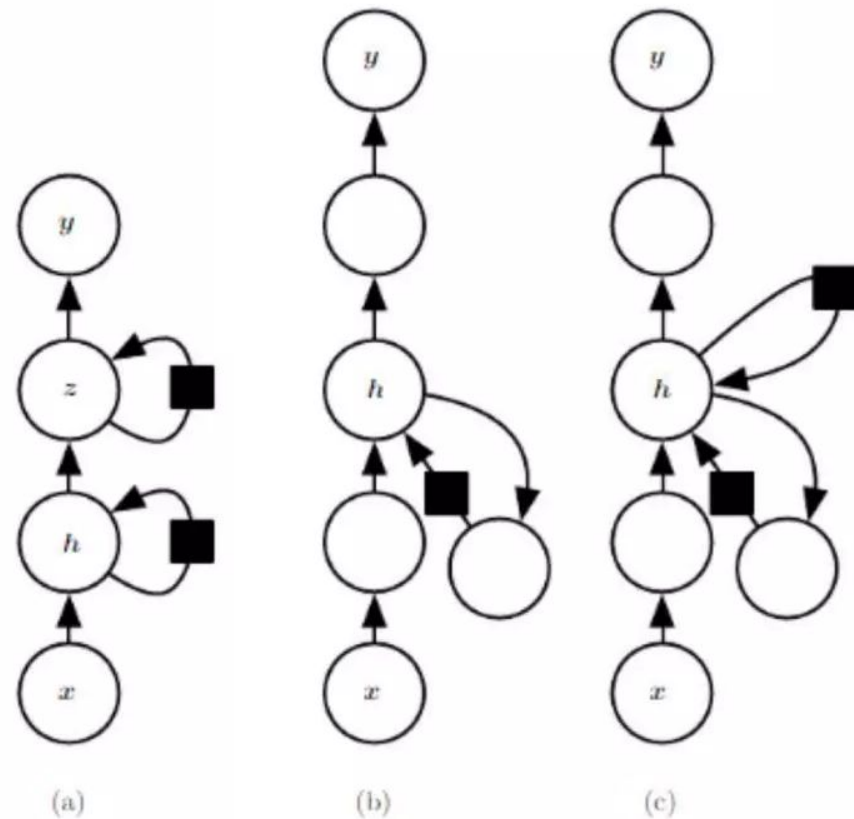


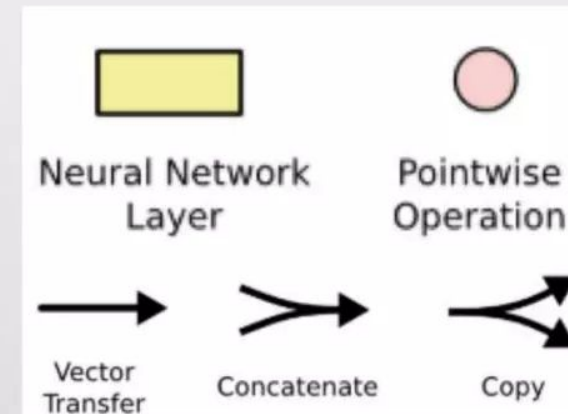
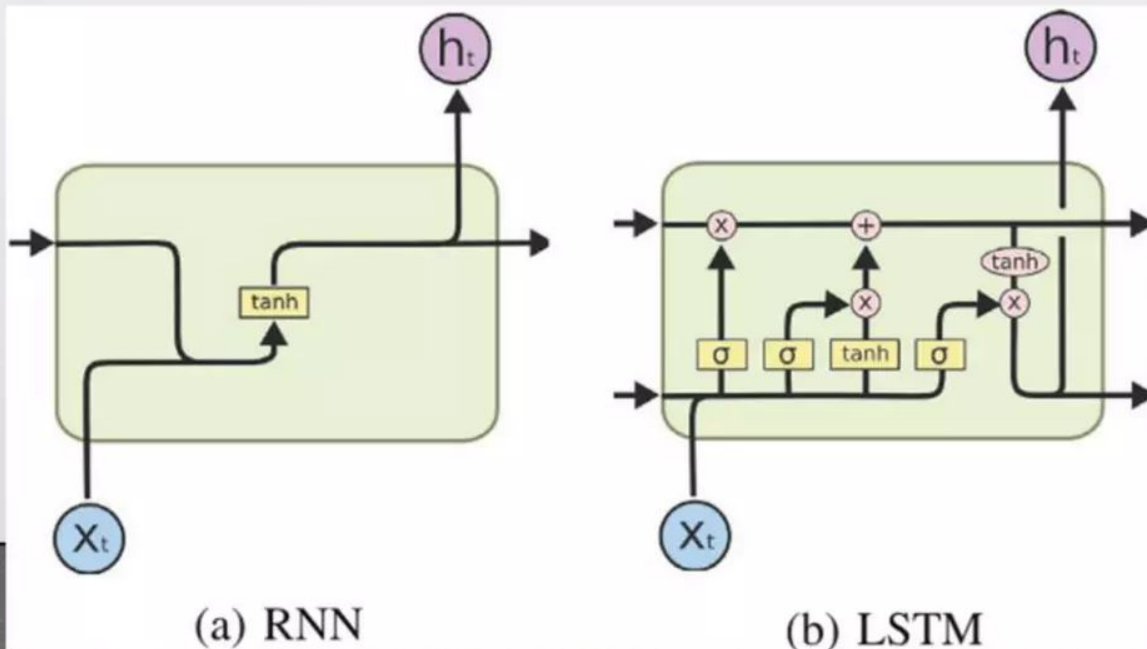
Figure 10.13: A recurrent neural network can be made deep in many ways (Pascanu *et al.*, 2014a). (a) The hidden recurrent state can be broken down into groups organized hierarchically. (b) Deeper computation (e.g., an MLP) can be introduced in the input-to-hidden, hidden-to-hidden, and hidden-to-output parts. This may lengthen the shortest path linking different time steps. (c) The path-lengthening effect can be mitigated by introducing skip connections.

Vanishing Gradient Problem

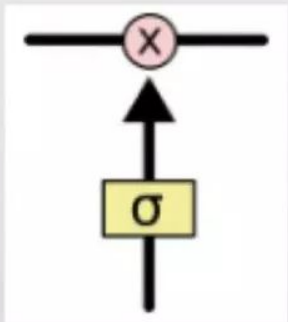
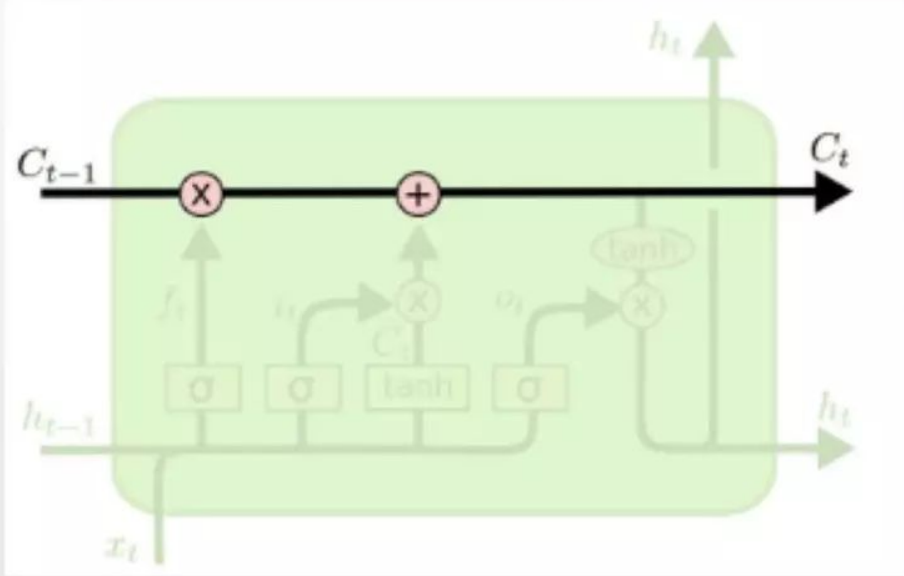
- Masing-masing bobot jaringan saraf menerima pembaruan yang sebanding dengan turunan parsial dari error function sehubungan dengan bobot saat ini di setiap iterasi pelatihan.
- Dalam beberapa kasus, gradien akan semakin kecil, secara efektif mencegah bobot berubah nilainya.
 - ❑ Dalam kasus terburuk, ini dapat sepenuhnya menghentikan jaringan saraf dari pelatihan lebih lanjut.
- Contoh:
 - ❑ fungsi aktivasi seperti fungsi tangen hiperbolik memiliki gradien dalam rentang $(0, 1)$, dan backpropagation menghitung gradien dengan aturan rantai.
 - ❑ Ini memiliki efek mengalikan n dari angka-angka kecil ini untuk menghitung gradien lapisan "depan" dalam jaringan lapisan- n
 - ✓ gradien (error signal) menurun secara eksponensial dengan n

Long Short Term Memory (LSTM) Network

- Diperkenalkan oleh **Hochreiter & Schmidhuber (1997)**, LSTM secara eksplisit dirancang untuk menghindari masalah ketergantungan jangka panjang.
- Jenis khusus RNN. Dalam RNN standar, modul berulang akan memiliki struktur yang sangat sederhana, seperti lapisan tanh tunggal. Modul berulang dalam LSTM berisi empat lapisan yang saling berinteraksi.



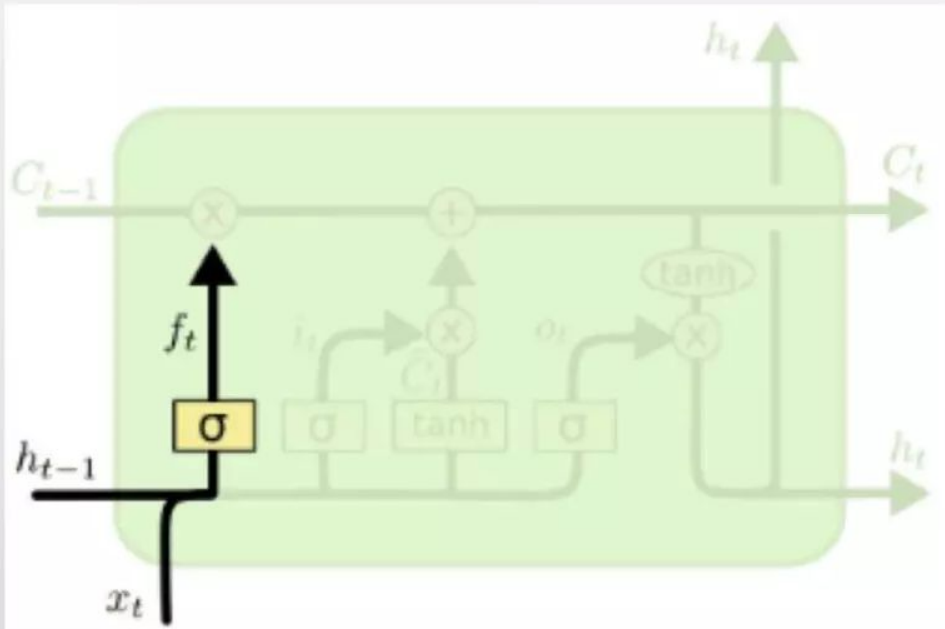
LSTM Unit



- Cell state c_t seperti conveyor belt. Ini berjalan lurus ke bawah seluruh rantai, dengan hanya beberapa interaksi linier kecil. Sangat mudah bagi informasi untuk mengalir begitu saja tanpa berubah.
- Gerbang adalah cara opsional untuk membiarkan informasi lewat. Lapisan sigmoid mengeluarkan angka antara nol dan satu, menjelaskan berapa banyak dari setiap komponen yang harus dilewati. Nilai nol berarti "jangan biarkan apa pun lewat", sedangkan nilai satu berarti "biarkan semuanya lewat!"

LSTM Unit: Forget Gate Layer

Forget gate layer: untuk memutuskan informasi apa yang akan kita **buang dari cell state**. Itu terlihat di h_{t-1} dan x_t , dan menampilkan angka antara 0 dan 1 untuk setiap angka dalam cell state C_{t-1} . Nilai 1 mewakili "sepenuhnya simpan ini" sementara nilai 0 mewakili "sepenuhnya singkirkan ini."

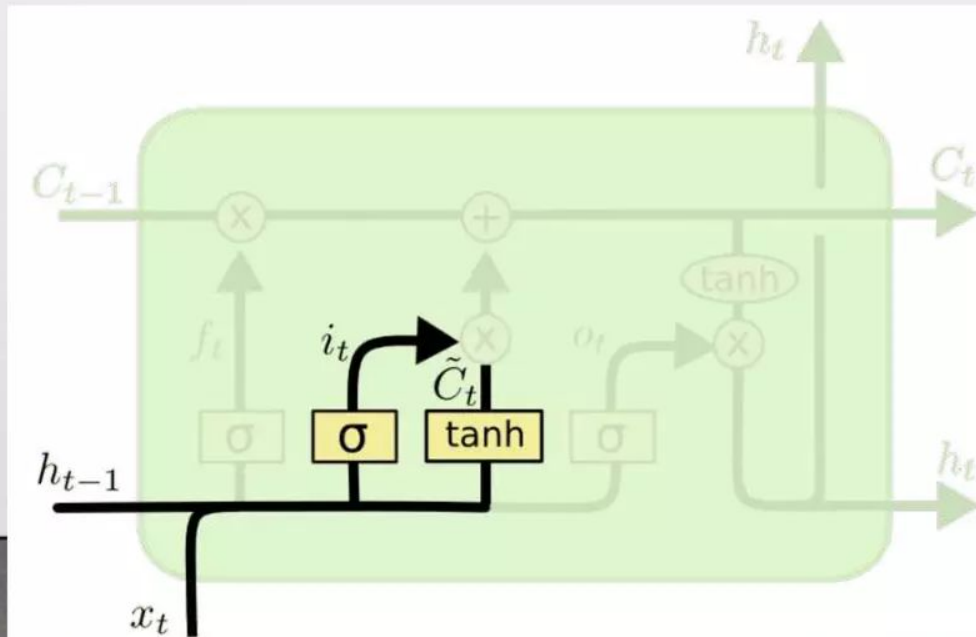


$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

LSTM Unit: Input Gate & Tanh Layer

Untuk memutuskan informasi baru apa yang akan kita simpan dalam cell state.

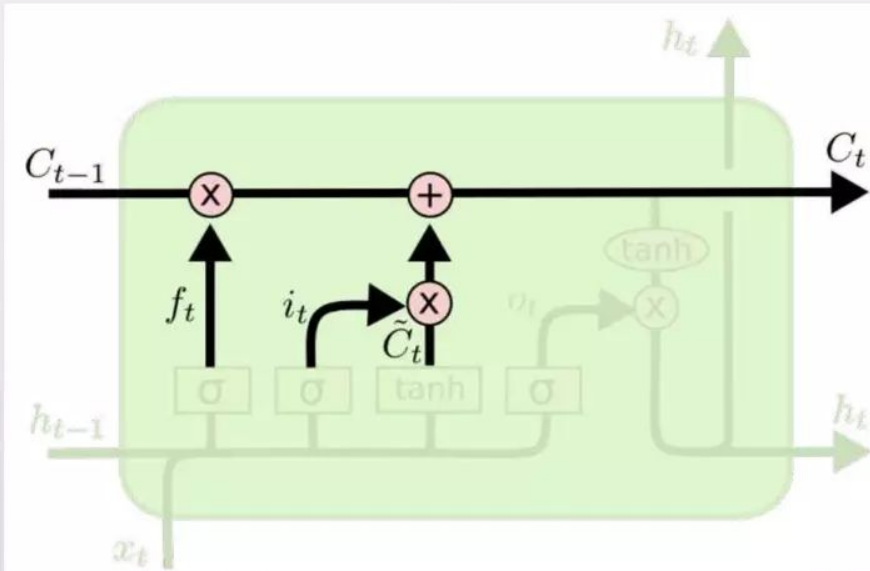
- Input gate layer memutuskan nilai mana yang akan kita perbarui;
- Tanh layer menciptakan vektor nilai kandidat baru, $\tilde{C}_t$, yang dapat ditambahkan ke state.



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTM Unit: Update C_{t-1} into C_t

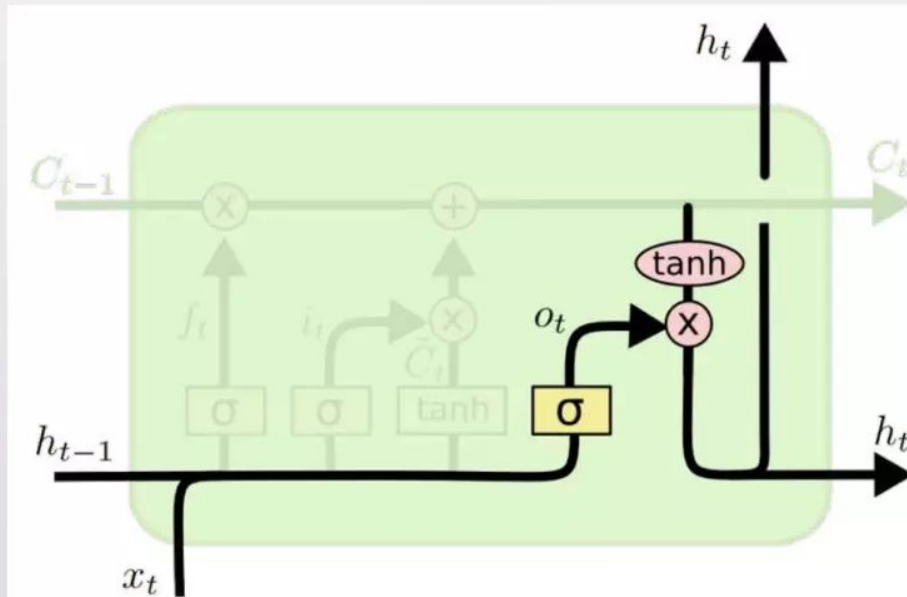
- Kalikan $C_{t-1} * f_t$, lupakan hal-hal yang telah diputuskan untuk dilupakan sebelumnya.
- Tambahkan $i_t * C_t$, nilai kandidat baru, yang diskalakan berdasarkan seberapa banyak kita memutuskan untuk memperbarui setiap state value.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM Unit: Output h_t

- Output h_t akan didasarkan pada C_t , tetapi akan menjadi versi yang difilter. Jalankan lapisan sigmoid yang memutuskan bagian mana dari status sel yang akan dihasilkan. Kemudian, masukkan C_t melalui tanh (untuk mendorong nilai menjadi antara -1 dan 1) dan kalikan dengan output dari gerbang sigmoid, sehingga kita hanya menampilkan bagian yang kita putuskan.



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Contoh LSTM pada Keras

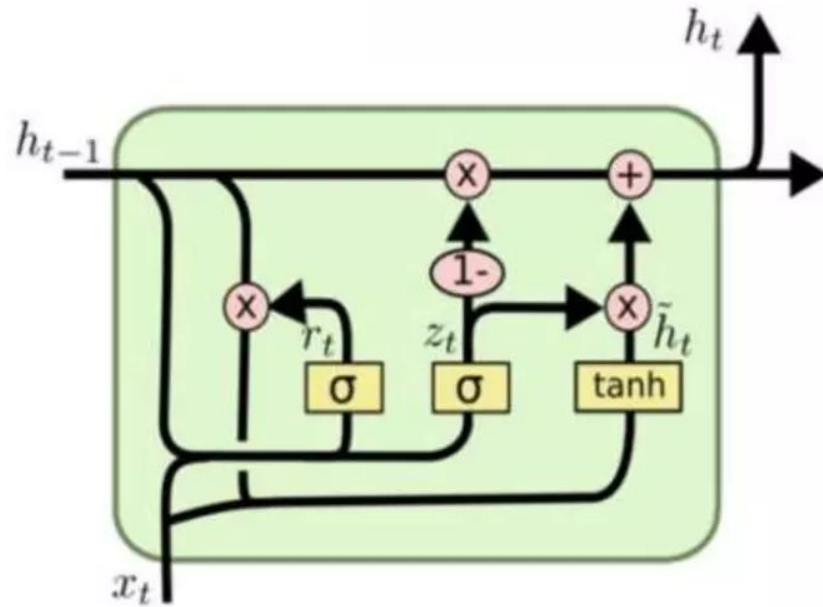


```
from keras.preprocessing import sequence
from keras.layers import Embedding
from keras import models
from keras import layers
model = models.Sequential()

maxlen = 500
embed_dim = 20

model.add(Embedding(jum_vocab+1, embed_dim, input_length=maxlen))
model.add(layers.LSTM(32))
model.add(layers.Dense(3, activation='softmax'))
model.compile(optimizer="adam",
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

Gated Recurrent Unit (GRU)



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

- Sebagai varian dari LSTM, GRU menggabungkan Forget Gate dan Input Gate ke dalam Update Gate tunggal.
 - Hal ini menggabungkan cell state dan hidden state dan juga menggabungkan perubahan lainnya.
 - Sebagai varian populer, model terakhir ini lebih sederhana daripada Model LSTM standar.
- [] menunjukkan proses concatenation;
 - * menunjukkan proses perkalian elemen;
 - . menunjukkan perkalian matriks.

TERIMA KASIH

//////
Ada pertanyaan ???

Minggu Depan → Eksplorasi Penerapan ANN dan LSTM